API Development - Day 3 of 3 (8 Hours)

API Testing, Performance, Observability, CI/CD & API Management

Simple guide for COMPLETE BEGINNERS (zero prior knowledge assumed)

Who this guide is for

This guide assumes you have never done API testing, never heard of load testing, never seen a dashboard or a trace, and have no internet. Every new tool and word is explained the first time it appears. Nothing is assumed.

How to read every topic

- What is it - explained in normal words, with a real-life example.
- Why we care - the one problem it solves.
- DO IT - exact file names, exact clicks, exact commands, complete code to paste.
- You should see - the exact result on screen. If your screen differs, stop and fix using the Troubleshooting appendix.
- STUDENT EXERCISE - short task students solve alone after the trainer demo.

Fixed rules for today

- All Day 3 code goes ONLY in: Controllers/Day3Controller.cs, Models/Day3.cs, Services/Day3/ folder.
- Your database Day3Db ALREADY exists with tables Users, ExamDetails, ExamResults and seed rows (admin, Ishan, Dev, Kavita; C# and WEB API exams). All Day-3 endpoints read/write THIS data.
- Docker containers needed: ONLY sqlserver + jaeger. Nothing else.
- Offline: no internet at any step. No new NuGet packages (testing and telemetry packages are already in the project).
- API always runs on http://localhost:5000 so every URL in this guide is exact.

Before we start - the 10 words you must know

Word	Simple meaning
API	A program that other programs talk to. Ours runs on http://localhost:5000.
Endpoint	One URL + one HTTP method = one job. Example: GET /api/day3/users.
Container (Docker)	A packed mini-computer: SQL Server and Jaeger run inside containers on your laptop, isolated from Windows.
SSMS	SQL Server Management Studio - the window where we LOOK at database tables.
Postman	Desktop app to send requests to our API (click Send instead of writing curl).
Swagger	Auto-made documentation page of our API at /swagger. Also lets you click "Try it out".
k6	A small program that pretends to be MANY users calling our API - used to test speed.
Jaeger	A website (localhost:16686) that draws a timed picture of every request - who called what and how long.
Test (xUnit)	A method that CHECKS another method and says Pass/Fail. `Assert.Equal(4, 2+2)` passes; expected 5 would fail and turn the suite red.
Migration	A script that changes database tables to match our C# classes. Day3Db already has migration "Day3InitialCreate" applied - we will not recreate it.

Module 0 - Start the offline environment (15 minutes)

Step 1 - start ONLY two containers

What is it: Docker Desktop runs the database and Jaeger as containers. We need just two today.

1. Internet OFF (Wi-Fi off / cable out). Everything from here works offline.
2. Start Docker Desktop. Wait for the whale icon to be steady.
3. Open PowerShell and type exactly:

```
cd "C:\Users\ishan\OneDrive\Desktop\Advanced API\OfflineSetup\Docker"
docker compose up -d sqlserver jaeger
docker compose ps
```

You should see:

```
In the list, sqlserver and jaeger show "Up". Others (keycloak, kong, prometheus,
grafana, otel-collector, wiremock) are OFF - that is ON PURPOSE today.
```

Step 2 - check the database exists (yes, already exists)

4. Open SSMS. Server name: localhost,1433. Login: sa. Password: Sa@123456.
5. Expand Databases -> Day3Db -> Tables.

You should see:

```
Tables visible: Users (4 rows: admin, Ishan, Dev, Kavita), ExamDetails (C#, WEB API),
ExamResults (2 rows). NOTHING to create. We only read and add data today.
```

Step 3 - run the API on the fixed port

```
cd "C:\Users\ishan\OneDrive\Desktop\Advanced API\Practicals"
dotnet run --urls http://localhost:5000
```

You should see:

```
Terminal shows: Now listening on: http://localhost:5000. Leave this window OPEN all
day.
```

6. Open Swagger: http://localhost:5000/swagger
7. Open Jaeger once: http://localhost:16686 (empty page is fine - Module 3 fills it)
8. Check k6: open a SECOND PowerShell and type: k6 version

You should see:

```
k6 version line appears. If not, ask the trainer - without k6 the performance demos
cannot run.
```

Step 4 - connect our API to the existing Day3Db

What is it: EF Core is the .NET-to-database translator. It needs entity classes (C# mirror of each table) and a DbContext (the connection object). We now create them so they match Day3Db EXACTLY.

File Practicals/Models/Day3.cs - replace everything with:

Note: class ExamUser is used instead of "User", because controllers already have a built-in property named User. Table names match through the DbSet names below.

```
using System.ComponentModel.DataAnnotations;

namespace Practicals.Models.Day3
{
    public class ExamUser               // <- matches table "Users"
    {
        public int Id { get; set; }
        [Required, MinLength(2), MaxLength(50)]
        public string Name { get; set; } = "";
        [Required, EmailAddress]
        public string Email { get; set; } = "";
        public string Password { get; set; } = "";   // teaching data only - real apps
hash this
        public string Role { get; set; } = "User";   // Admin / User / Student
    }

    public class ExamDetail             // <- matches table "ExamDetails"
    {
        public int Id { get; set; }
        [Required, MinLength(2)]
        public string ExamName { get; set; } = "";
        public DateTime ExamDate { get; set; }
        public string ExamLocation { get; set; } = "";
    }

    public class ExamResult             // <- matches table "ExamResults"
    {
        public int Id { get; set; }
        public int UserId { get; set; }
        public int ExamId { get; set; }
        [Range(0, 100)]
        public decimal Score { get; set; }
    }
}
```

File Practicals/Services/Day3/Day3DbContext.cs - create:

```
using Microsoft.EntityFrameworkCore;
using Practicals.Models.Day3;

namespace Practicals.Services.Day3
{
    public class Day3DbContext : DbContext
    {
        public Day3DbContext(DbContextOptions<Day3DbContext> o) : base(o) { }
```

```
        public DbSet<ExamUser> Users => Set<ExamUser>();          // <- DbSet name =
table name "Users"
        public DbSet<ExamDetail> ExamDetails => Set<ExamDetail>();
        public DbSet<ExamResult> ExamResults => Set<ExamResult>();
    }
}
```

appsettings.json - add this (keep other settings untouched):

```
"ConnectionStrings": {
  "Day3Db": "Server=localhost,1433;Database=Day3Db;User
Id=sa;Password=Sa@123456;TrustServerCertificate=True"
}
```

Program.cs - add BEFORE the line var app = builder.Build();

```
builder.Services.AddDbContext<Practicals.Services.Day3.Day3DbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Day3Db")));
builder.Services.AddMemoryCache();      // needed by caching + idempotency demos
builder.Services.AddHttpClient();       // needed by partner-call demos
```

IMPORTANT: do NOT run Update-Database. The database already has these tables and data. We are only teaching the API to TALK to it.

First endpoint reading the REAL data - replace Controllers/Day3Controller.cs with:

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Practicals.Models.Day3;
using Practicals.Services.Day3;

namespace Practicals.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class Day3Controller : ControllerBase
    {
        private readonly Day3DbContext _db;
        private readonly ILogger<Day3Controller> _log;
        public Day3Controller(Day3DbContext db, ILogger<Day3Controller> log)
        { _db = db; _log = log; }

        // GET /api/day3/health - always fast; answers "are you alive?"
        [HttpGet("health")]
        public IActionResult Health() => Ok("OK");

        // GET /api/day3/users - all exam users from the REAL database
        [HttpGet("users")]
        public async Task<IActionResult> Users()
            => Ok(await _db.Users.AsNoTracking().ToListAsync());

        // GET /api/day3/users/2 - one user or 404
        [HttpGet("users/{id:int}")]
```

```
        public async Task<IActionResult> UserById(int id)
        {
            var u = await _db.Users.FindAsync(id);
            return u is null ? NotFound(new { error = $"user {id} not found" }) :
Ok(u);
        }

        // GET /api/day3/exams - all exams
        [HttpGet("exams")]
        public async Task<IActionResult> Exams()
            => Ok(await _db.ExamDetails.AsNoTracking().ToListAsync());
    }
}
```

9. Save all files. The API auto-reloads (or Ctrl+C, dotnet run again).
10. Postman: GET http://localhost:5000/api/day3/users

You should see:

```
HTTP 200 with 4 users: admin (id1), Ishan (id2), Dev (id3), Kavita (id4).
GET /api/day3/users/2 -> Ishan.
GET /api/day3/users/99 -> 404 {"error":"user 99 not found"}
```

STUDENT EXERCISE - Module 0 - first touch

11. In Swagger, execute GET /api/day3/exams yourself. Write down the status code and how many exams came back.
12. On paper, say in one sentence what each of these tools does: SQL Server, Jaeger, k6, Postman, Swagger.

Module 1 - API Quality & Testing (9:30 - 11:30)

1.1 What is testing? (first, for someone who has never tested)

What is it: A test is a small program that CHECKS another program and answers a simple question: pass or fail. Example: you wrote a method that adds two numbers; a test calls it with 2 and 2 and checks the answer is 4. If someday someone breaks the method, the test turns RED and warns you before users notice.

Why we care: APIs serve other people's apps. One broken endpoint breaks THEIR work. Tested APIs can change fast without fear.

Mini demo on paper: a test in our project looks like Assert.Equal(200, (int)response.StatusCode). If the answer is anything else, dotnet test prints FAILED. That is all a test is.

1.2 API testing strategy - the pyramid

What is it: Different bugs need different tests. We arrange them like a pyramid: a LOT of fast cheap tests at the bottom (unit), some medium tests in the middle (integration), a few slow realistic tests at the top (end-to-end). One endpoint deserves checks at several layers.

Layer	Simple question	Example today
Unit (many)	is one method right?	score 0-100 rule checks
Integration (some)	does real HTTP + real DB answer right?	GET /users/2 returns Ishan
End-to-end (few)	can a student complete a whole journey?	postman collection runs 3 steps
Contract	did we break what callers use?	openapi file still has the paths
Load (few)	does it survive many users?	k6 in Module 2

1.3 Unit and integration tests - first tests you will write

What is it: Unit test: calls ONE c# method, no server, no db - 5 milliseconds. Integration test: starts the WHOLE api in memory and hits it like a real client, including the real SQL Server - 1 second. Both run with the same command: dotnet test.

DO IT - Step 1: one line in Program.cs

Add this ONE line at the very END of Program.cs (tests need it):

```
public partial class Program { }
```

DO IT - Step 2: create folder Practicals/Tests/Day3 and file ModelTests.cs

```
using System.ComponentModel.DataAnnotations;
using Practicals.Models.Day3;
using Xunit;

namespace Practicals.Tests.Day3
{
    public class ModelTests
```

```
    {
        // [Fact] = a test with no inputs
        [Fact]
        public void Valid_result_is_accepted()
        {
            var list = Validate(new ExamResult { UserId = 3, ExamId = 1, Score = 85 });
            Assert.Empty(list);   // no errors expected
        }

        // [Theory] = ONE test, MANY input values
        [Theory]
        [InlineData(-1, true)]     // below 0 -> INVALID
        [InlineData(0, false)]     // exactly 0 -> valid
        [InlineData(100, false)]   // exactly 100 -> valid
        [InlineData(101, true)]    // above 100 -> INVALID
        public void Score_must_be_between_0_and_100(double score, bool expectBad)
        {
            var isBad = Validate(new ExamResult { UserId = 3, ExamId = 1, Score =
(decimal)score }).Any();
            Assert.Equal(expectBad, isBad);
        }

        private static IList<ValidationResult> Validate(ExamResult r)
        {
            var list = new List<ValidationResult>();
            Validator.TryValidateObject(r, new ValidationContext(r), list, true);
            return list;
        }
    }
}
```

13. PowerShell (second window, Practicals folder): dotnet test Practicals/Practicals.csproj --filter ModelTests

You should see:

```
Passed! - Failed: 0, Passed: 5. Value 101 FAILED the rule and the test checked it would
- both sides of the boundary are proven.
```

DO IT - Step 3: integration tests - file ApiTests.cs

```
using System.Net;
using Microsoft.AspNetCore.Mvc.Testing;
using Practicals.Models.Day3;
using Xunit;

namespace Practicals.Tests.Day3
{
    public class ApiTests : IClassFixture<WebApplicationFactory<Program>>
    {
        private readonly HttpClient _c;
        public ApiTests(WebApplicationFactory<Program> f) => _c = f.CreateClient();
```

```
        [Fact]
        public async Task Health_returns_200()
            => Assert.Equal(HttpStatusCode.OK,
                (await _c.GetAsync("/api/day3/health")).StatusCode);

        [Fact]
        public async Task User_2_is_Ishan()
        {
            var u = await _c.GetFromJsonAsync<ExamUser>("/api/day3/users/2");
            Assert.Equal("Ishan", u!.Name);          // real seeded row from YOUR db
        }

        [Fact]
        public async Task Unknown_user_gives_404_not_500()
            => Assert.Equal(HttpStatusCode.NotFound,
                (await _c.GetAsync("/api/day3/users/99999")).StatusCode);
    }
}
```

14. Run: dotnet test Practicals/Practicals.csproj

You should see:

```
Passed: 8 (5 unit + 3 integration). The tests talked to the REAL database - that is why
user 2 = Ishan works.
```

STUDENT EXERCISE - Tests

15. Add one [Theory] row yourself: score 50 must be valid (expectBad=false). Run dotnet test again and see the count go 5 -> 6.
16. Break one test on purpose (expect valid for 101), run, see the red output, then fix it back.

1.4 End-to-end testing - a real student journey

What is it: End-to-end = a full story of a real user, step by step, against the running api. We make a Postman collection: check the user exists -> post an exam result -> read back totals.

Add the result endpoint first (Day3Controller, paste before the last } of the class):

```
// POST /api/day3/results   header: Idempotency-Key
[HttpPost("results")]
public async Task<IActionResult> SubmitResult(ExamResult input,
    [FromHeader(Name = "Idempotency-Key")] string? key)
{
    if (string.IsNullOrWhiteSpace(key))
        return BadRequest(new { error = "Idempotency-Key header required" });

    var existing = await _db.ExamResults.FirstOrDefaultAsync(
        r => r.UserId == input.UserId && r.ExamId == input.ExamId);
    if (existing is not null)           // same student + same exam = replay, no
duplicate
    {
        Response.Headers["X-Replay"] = "true";
```

```
        return Ok(existing);
    }

    if (await _db.Users.FindAsync(input.UserId) is null)
        return NotFound(new { error = "user not found" });
    if (await _db.ExamDetails.FindAsync(input.ExamId) is null)
        return NotFound(new { error = "exam not found" });

    input.Id = 0;
    _db.ExamResults.Add(input);
    await _db.SaveChangesAsync();
    _log.LogInformation("Result saved: user {UserId} exam {ExamId} score {Score}",
        input.UserId, input.ExamId, input.Score);
    return Ok(input);
}
```

E2E collection in Postman - create collection "Day3-E2E", 3 requests in order:

#	Request	Tests tab
1	GET http://localhost:5000/api/day3/users/3	pm.expect(pm.response.json().name).to.eql("Dev");
2	POST /api/day3/results with header Idempotency-Key: e2e-1 and body {"userId":2,"examId":1,"score":85}	pm.expect(pm.response.code).to.eql(200);
3	GET /api/day3/results-e2e-check (reader endpoint below)	pm.expect(pm.response.json().count).to.be.gte(2);

17. Add the small reader endpoint (Day3Controller):

```
// GET /api/day3/results-e2e-check - how many result rows exist
[HttpGet("results-e2e-check")]
public async Task<IActionResult> ResultCount()
    => Ok(new { count = await _db.ExamResults.CountAsync() });
```

18. Click the collection name -> Run collection -> watch it run all 3 steps.

You should see:

```
3 green checks. Step 1 READ, step 2 WROTE (Ishan, exam C#, 85), step 3 PROVED the
write. That journey is an end-to-end test. Keep the count of e2e tests small - they are
the slowest layer.
```

1.5 Consumer-driven contract testing

What is it: Imagine a mobile app uses our /api/day3/users list and reads fields "id", "name", "email". The consumer writes its needs in a file; WE keep proving our API still satisfies it. If someone renames "email" to "mail" tomorrow, the safety net fails BEFORE the mobile app does.

Create file Practicals/Tests/Day3/contract.json

```
{ "consumer": "ResultBoardApp",
  "paths": ["/api/Day3/users", "/api/Day3/users/{id}", "/api/Day3/results"],
  "userFields": ["id", "name", "email", "role"] }
```

Create file Practicals/Tests/Day3/ContractTests.cs

```
using Microsoft.AspNetCore.Mvc.Testing;
using Xunit;

namespace Practicals.Tests.Day3
{
    public class ContractTests : IClassFixture<WebApplicationFactory<Program>>
    {
        [Fact]
        public async Task Spec_still_contains_what_the_consumer_needs()
        {
            var spec = await new WebApplicationFactory<Program>().CreateClient()
                .GetStringAsync("/openapi/v1.json");     // the LIVE contract of the
API
            foreach (var path in new[] { "/api/Day3/users", "/api/Day3/users/{id}",
"/api/Day3/results" })
                Assert.Contains(path, spec);
            foreach (var field in new[] { "id", "name", "email", "role" })
                Assert.Contains($"\"{field}\"", spec);
        }
    }
}
```

19. Run: dotnet test --filter ContractTests
20. Feel the power: in Models/Day3.cs temporarily rename ExamUser.Email to Mail, run the test again, then rename back.

You should see:

```
Pass first; after the rename - Failed (word "email" missing from the spec). That prove-
it-yourself moment IS consumer-driven contract testing.
```

1.6 Positive and negative test scenarios

What is it: For every endpoint, ask two questions. Positive: valid request -> correct answer? Negative: bad request -> clean 4xx, never a crash (500)?

Walk this matrix in Postman (everything already coded - just CALL them):

Type	Call	Must answer
positive	GET /api/day3/users/2	200 + Ishan
positive	POST /api/day3/results with a fresh key	200 + saved row
negative	POST /api/day3/results without the key header	400 key required

negative	POST results for user 999	404 user not found
negative	POST results with score 999	400 (Range rule failed)
negative	GET /api/day3/users/abc	404 (the :int route did not match)

Rule to remember: a 5xx reply to bad input means OUR code is not defensive enough. Fresher first audit: TRY to break the API with bad input. Todays goal - you cannot create a 500 by typing bad data.

1.7 Boundary and concurrency testing

What is it: Boundary bugs live at the EDGES of rules (score 0 and 100 are ok; -1 and 101 are not - we proved those in ModelTests). Concurrency bugs hide between two simultaneous requests - that is where duplicates and wrong totals come from. We test concurrency with a real storm: 20 identical requests at the same time.

Create folder Practicals/k6 and file k6/samekey.js:

k6 is a program that acts as many users at once. This file says: 20 callers, same instant, SAME idempotency key.

```
import http from 'k6/http';

export const options = { vus: 20, iterations: 20 };   // 20 parallel callers

export default function () {
  http.post('http://localhost:5000/api/day3/results',
    JSON.stringify({ userId: 4, examId: 1, score: 88 }),   // Kavita, exam C#
    { headers: { 'Content-Type': 'application/json',
                 'Idempotency-Key': 'kavita-cs-storm' } });
}
```

21. Before: GET /api/day3/results-e2e-check - note the count.
22. Run: k6 run k6/samekey.js
23. After: call the check again.

You should see:

```
Count increased by EXACTLY ONE. 20 parallel calls, 1 row. k6 summary shows 20 requests,
0 failed - and still only one result exists. (Kavita already has a WEB API result in
the seed; we chose C# to keep this visible.)
```

1.8 Idempotency testing

What is it: Idempotent = repeating the same action gives the SAME result and no duplicates. Mobile networks retry without asking the user; without idempotency, Kavita would end up with two rows in the same exam. Testing idempotency = call twice, compare answers, count rows.

24. Postman: POST /api/day3/results header Idempotency-Key: dev-webapi body {"userId":3,"examId":2,"score":91} - press SEND twice. Note the "id" in both responses.

You should see:

```
Both 200 with the SAME id. Second response has header X-Replay: true. SSMS: SELECT *
FROM ExamResults WHERE UserId=3 AND ExamId=2 -> exactly ONE row.
```

STUDENT EXERCISE - Idempotency and concurrency

25. Edit k6/samekey.js to use your OWN fresh key and a user/exam pair with no seed row, run it, then prove in SSMS with SELECT COUNT(*) that exactly one row exists for your key pair.
26. Explain in two sentences why the count did not grow by 20.

1.9 Test data and mocking

What is it: Rule 1 - tests own their data: only depend on the CONTROLLED seed (we know user 2 is Ishan because the migration inserted it). Rule 2 - mocking: when a unit test needs a collaborator, hand it a simple hand-made FAKE instead of the real thing. No extra packages needed.

Demo the hand-made fake - create Services/Day3/GradeRule.cs:

```
namespace Practicals.Services.Day3
{
    public interface IGradeRule { string Grade(decimal score); }

    public class GradeRule : IGradeRule
    {
        public string Grade(decimal s) => s >= 80 ? "A" : s >= 50 ? "B" : "C";
    }

    public class FakeGradeRule : IGradeRule   // the "mock": always answers A, zero
logic
    {
        public string Grade(decimal s) => "A";
    }
}
```

27. Add two tests at the bottom of ModelTests.cs:

```
[Fact]
public void Fake_rule_always_gives_A()
{
    IGradeRule fake = new FakeGradeRule();
    Assert.Equal("A", fake.Grade(10));
}

[Fact]
public void Real_rule_gives_B_for_mid_scores()
{
    IGradeRule real = new GradeRule();
    Assert.Equal("B", real.Grade(60));
}
```

You should see:

```
Both pass (total tests grow to 10). One used the fake - proves the harness works; one
used the real rule - proves the logic. THAT is mocking without any library.
```

1.10 Service virtualization (our fake partner)

What is it: Sometimes an external system is unavailable - no internet, vendor down, or not built yet. We create a VIRTUAL one: a small endpoint inside our own API pretending to be the external service. It can answer OK, slowly, or with an error - on demand.

Add to Day3Controller:

```
// GET /mock/reportcard?mode=ok|slow|fail   (pretend: the "Result Publishing Service")
[HttpGet("/mock/reportcard")]
public async Task<IActionResult> FakePartner([FromQuery] string mode = "ok")
{
    if (mode == "slow") await Task.Delay(4000);                              // slow
vendor
    if (mode == "fail") return StatusCode(503, new { error = "broken" });    // broken
vendor
    return Ok(new { published = true, by = "Fake ReportCard Vendor" });      // happy
vendor
}
```

28. In Postman call /mock/reportcard, then ?mode=slow, then ?mode=fail.

You should see:

```
Instant 200, then ~4-second 200, then 503. You now own a controllable third party.
Modules 2 and 3 reuse these three modes for latency and failure drills.
```

1.11 OpenAPI validation and compatibility checks

What is it: /openapi/v1.json is the machine-readable list of everything our API offers. Validation = the file is well-formed (Swagger opening on it proves that). Compatibility = whatever callers used yesterday must still exist today.

29. Freeze todays contract:

```
curl http://localhost:5000/openapi/v1.json -o Practicals/Tests/Day3/openapi-
baseline.json
```

ContractTests from 1.5 already compare future code against this contract on every run. Wall rule: ADDING a field or endpoint = compatible; RENAMING, REMOVING or changing a type = breaking (needs a new version - Module 5).

1.12 Automated quality gates

What is it: One command must decide go/no-go: dotnet test. Any red = nothing ships. Coverage tells us HOW MUCH code the tests touched (so "we have tests" cannot be a lie).

30. Run the full gate:

```
dotnet test Practicals/Practicals.csproj --collect:"XPlat Code Coverage"
```

You should see:

```
Passed summary, plus a file TestResults/<guid>/coverage.cobertura.xml - open it: line
near the top shows line-rate="0.xx", your measured coverage. Module 4 turns this into 5
named gates behind one command.
```

STUDENT EXERCISE - Module 1 wrap

31. Write one Fact that calls GET /api/day3/exams and asserts status 200 AND that the body contains "WEB API".
32. Find one more negative case for POST /results that is NOT in the 1.6 table (hint: completely empty body) - prove the answer is 400, never 500.

Module 2 - Performance & Resilience (11:30 - 13:00)

2.1 Latency, throughput and concurrency

What is it: Three words, one chai stall. Latency = how long ONE customer waits (order to chai in hand). Throughput = customers served per minute. Concurrency = how many customers are waiting RIGHT NOW. When the queue grows beyond a point, each customer waits longer (latency rises) BEFORE the stall actually fails.

Why we care: Users do not abandon fast APIs. Measuring these three correctly is step one; improving blindly is guessing.

Demo - our first load test: create folder Practicals/k6 and file k6/load.js (complete):

This tells k6: slowly grow to 20 users, hold for 1 minute, cool down. Each user asks for the user list again and again.

```
import http from 'k6/http';
import { check, sleep } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 20 },   // 30s to reach 20 users
    { duration: '1m',  target: 20 },   // hold 20 users
    { duration: '20s', target: 0  },   // cool down
  ],
  thresholds: { http_req_duration: ['p(95)<300'] },   // 95% within 300ms
};

export default function () {
  const res = http.get('http://localhost:5000/api/day3/users');
  check(res, { 'status 200': (r) => r.status === 200 });
  sleep(1);   // 1s between calls, like a human
}
```

33. Run: k6 run k6/load.js
34. Read the summary numbers together as a class.

You should see:

```
k6 prints the summary: p(95) just a few ms, checks 100% passed, thresholds PASSED in
green. You just ran a load test. Teachers: point at exactly three numbers - p(95) is
latency, http_reqs/s is throughput, vus is concurrency (20).
```

2.2 Capacity planning fundamentals

What is it: Capacity planning answers one management question: "when exam-day traffic comes, will we survive?" Recipe: measure your ceiling (2.4 does that), estimate the expected peak (business guess), require comfortable headroom (around 2x).

One-sentence plan - fill it AFTER the 2.4 run:

```
"This API serves about ____ requests/sec with p95 under 300 ms on our machine.
 Our expected exam-day peak is ____ requests/sec.  Headroom = ____ x."
```

2.3 Load and stress testing

What is it: Load test = normal busy day, sustained (load.js above). Stress test = push BEYOND expected until the API bends - we learn WHERE it bends and whether it recovers when traffic falls.

Create k6/stress.js:

```
import http from 'k6/http';
import { check } from 'k6';

export const options = {
  stages: [
    { duration: '30s', target: 20 },
    { duration: '30s', target: 60 },
    { duration: '30s', target: 150 },   // deliberately beyond a laptop-API comfort
zone
    { duration: '20s', target: 0 },
  ],
};

export default function () {
  check(http.get('http://localhost:5000/api/day3/users'), { ok: r => r.status === 200
});
}
```

35. Run: k6 run k6/stress.js - WATCH the live p(95) number during each stage.

You should see:

```
p(95) stays low at 20-60 users; during the heaviest stage it climbs (exact jump depends
on your laptop). After ramp-down the numbers recover = the system is elastic. The climb
point is what we measured.
```

2.4 Spike, soak and breakpoint (the three special shapes)

Test	Shape	Question it answers
Spike	5 users -> suddenly 300 -> back to 5	survive a sudden flood? settle back?
Soak	25 users for 10 minutes	does memory slowly creep? (leak check)
Breakpoint	demand grows +100 rps every 30s	the exact ceiling where failures start

Create the three scripts - complete files:

```
----- file k6/spike.js -----
import http from 'k6/http';
```

```
export const options = { stages: [
  { duration: '20s', target: 5 },
  { duration: '30s', target: 300 },
  { duration: '20s', target: 5 } ] };
export default function () { http.get('http://localhost:5000/api/day3/users'); }

----- file k6/soak.js -----
import http from 'k6/http';
export const options = { vus: 25, duration: '10m' };
export default function () { http.get('http://localhost:5000/api/day3/users'); }

----- file k6/breakpoint.js -----
import http from 'k6/http';
export const options = {
  executor: 'ramping-arrival-rate',
  startRate: 100, timeUnit: '1s', preAllocatedVUs: 500,
  stages: [ ...Array(8) ].map((_, i) =>
    ({ duration: '30s', target: 100 * (i + 1) })),  // 100, 200 ... 800 rps
};
export default function () { http.get('http://localhost:5000/api/day3/users'); }
```

36. Run each. After breakpoint, note the FIRST stage where http_req_failed left 0%.
37. During soak, open Task Manager on the dotnet process and watch its memory.

You should see:

```
Spike: numbers jump then recover. Soak: memory line flat = healthy. Breakpoint: ceiling
found (a few hundred rps on a classroom laptop) - write THAT ceiling into the 2.2
sentence. Capacity planning is now measured, not guessed.
```

2.5 Application and database profiling

What is it: Profiling = measuring WHERE time goes: our code, or the database. Two beginner lenses: EF logging (prints every SQL statement with its time into the API terminal) and Task Manager (CPU/RAM of dotnet).

38. Upgrade the DbContext registration in Program.cs (replace what you added in Module 0):

```
builder.Services.AddDbContext<Practicals.Services.Day3.Day3DbContext>(o =>
    o.UseSqlServer(builder.Configuration.GetConnectionString("Day3Db"))
     .LogTo(Console.WriteLine, Microsoft.Extensions.Logging.LogLevel.Information));
```

39. Restart the API. GET /api/day3/users once.

You should see:

```
The terminal prints the real SQL - SELECT [u].[Id], [u].[Name] ... FROM [Users] with
Executed DbCommand (X ms). That printout IS database profiling. (Extra: SSMS -> right-
click server -> Activity Monitor while k6 runs - live sessions.)
```

2.6 Identifying API bottlenecks

What is it: The bottleneck game: same machine, same load, change ONLY the endpoint. If /health is fast but another endpoint is slow, that endpoint path is guilty - not the server, not k6.

40. Add the delay endpoint to Day3Controller:

```
// GET /api/day3/slow?ms=800  (pretend: a slow query or slow partner)
[HttpGet("slow")]
public async Task<IActionResult> Slow([FromQuery] int ms = 800)
{
    await Task.Delay(ms);
    return Ok(new { delayedMs = ms });
}
```

41. Point load.js at /api/day3/health -> note p(95). Then point it at /api/day3/slow?ms=800 -> note p(95).

Endpoint	rough p95	conclusion
/health	a few ms	machine and framework are fine
/slow?ms=800	about 800 ms	the delay INSIDE this endpoint is the bottleneck

You should see:

```
Two numbers on the board give a diagnosis with zero guessing: "api slow" became "this
endpoint path is slow".
```

2.7 Caching and batching

What is it: Caching: keep the just-computed answer in memory a short time instead of asking the database every time. Batching: one request carrying 20 things instead of 20 separate requests. Both cut repeated work.

Add to Day3Controller:

```
// GET /api/day3/users-cached   (database asked at most once per 30 seconds)
[HttpGet("users-cached")]
public async Task<IActionResult> UsersCached(
    [FromServices] Microsoft.Extensions.Caching.Memory.IMemoryCache cache)
{
    var users = await cache.GetOrCreateAsync("all-users", async entry =>
    {
        entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromSeconds(30);
        return await _db.Users.AsNoTracking().ToListAsync();
    });
    return Ok(users);
}

// POST /api/day3/results-batch   (insert MANY results in ONE call)
```

```
[HttpPost("results-batch")]
public async Task<IActionResult> Batch(List<ExamResult> items)
{
    _db.ExamResults.AddRange(items);
    await _db.SaveChangesAsync();
    return Ok($"inserted {items.Count}");
}
```

42. With EF logging on: call /api/day3/users three times - count the SELECT prints (3). Now call /api/day3/users-cached three times.

You should see:

```
Cached endpoint printed only ONE SELECT (the very first call); calls 2 and 3 came from
memory. Batch: POST a JSON array of two results -> one call says "inserted 2".
```

2.8 Compression and pagination

What is it: Compression: shrink the network package (gzip) - free speed-up for JSON responses. Pagination: never return "everything" - client asks for page N of size M, so answers stay small even with a million rows.

43. Compression: in Program.cs add BEFORE Build: builder.Services.AddResponseCompression(); and AFTER Build: app.UseResponseCompression(); Restart.
44. Postman: GET /api/day3/users with header Accept-Encoding: gzip.

You should see:

```
Response has header Content-Encoding: gzip - the body travelled compressed.
```

Pagination endpoint:

```
// GET /api/day3/users-page?page=2&pageSize=2
[HttpGet("users-page")]
public async Task<IActionResult> UsersPage(int page = 1, int pageSize = 10)
{
    pageSize = Math.Clamp(pageSize, 1, 50);     // hard ceiling
    var q = _db.Users.AsNoTracking().OrderBy(u => u.Id);
    return Ok(new
    {
        page, pageSize,
        total = await q.CountAsync(),
        items = await q.Skip((page - 1) * pageSize).Take(pageSize)
                       .Select(u => new { u.Id, u.Name }).ToListAsync()
    });
}
```

You should see:

```
page=1&pageSize=2 -> admin, Ishan (total: 4). page=2 -> Dev, Kavita. The terminal SQL
shows OFFSET/FETCH - the DATABASE paged, not memory.
```

2.9 Asynchronous processing and offloading

What is it: A request must never WAIT for slow work. Pattern: POST creates the job -> the API answers 202 Accepted immediately with the job id -> a hidden background worker does the work -> the caller asks GET /jobs/{id} for queued/processing/done.

Create Services/Day3/JobService.cs:

```
using System.Collections.Concurrent;

namespace Practicals.Services.Day3
{
    public record Job(string Id, string Status);

    public class JobService
    {
        private readonly ConcurrentDictionary<string, Job> _jobs = new();
        public Job Submit()
        {
            var j = new Job(Guid.NewGuid().ToString("N")[..8], "queued");
            _jobs[j.Id] = j;
            _ = Task.Run(async () =>
            {
                _jobs[j.Id] = j with { Status = "processing" };
                await Task.Delay(3000);                      // the slow work
                _jobs[j.Id] = j with { Status = "done" };
            });
            return j;
        }
        public Job? Get(string id) => _jobs.TryGetValue(id, out var j) ? j : null;
    }
}
```

45. Register in Program.cs (with the other builder.Services lines):
    builder.Services.AddSingleton<Practicals.Services.Day3.JobService>();
46. Add endpoints to Day3Controller:

```
// POST /api/day3/jobs  -> 202 Accepted immediately
[HttpPost("jobs")]
public IActionResult NewJob([FromServices] JobService js)
{
    var j = js.Submit();
    return AcceptedAtAction(nameof(GetJob), new { id = j.Id }, j);
}

// GET /api/day3/jobs/{id}  -> job status
[HttpGet("jobs/{id}")]
public IActionResult GetJob(string id, [FromServices] JobService js)
    => js.Get(id) is { } j ? Ok(j) : NotFound();
```

47. POST /api/day3/jobs; immediately follow the Location value from the response. Wait 3 seconds, GET it again.
48. Then fire 10 POSTs while a k6 load test is still running against /users.

You should see:

```
POST answered in milliseconds with 202 + Location. The job status went queued ->
processing -> done without anyone waiting. And under load, job submission stayed
instant - the slowness moved into the background.
```

STUDENT EXERCISE - Module 2 wrap

49. Point load.js at /slow?ms=300 - the threshold still passes? Find the biggest ms where p(95)<300 stays green.
50. Call /users-page?pageSize=99999 - prove the cap forces 50 rows maximum.

Module 3 - API Observability & Diagnostics (13:45 - 15:15)

What is it: Observability = seeing INSIDE a running program from the signals it sends out, without stopping it or attaching a debugger. Three friends work together: LOGS (what happened, line by line), METRICS (countable numbers: how many, how fast, how many failed), TRACES (a timed picture of ONE request - every step it took). Today: logs -> console as JSON; metrics -> a tiny endpoint we build ourselves; traces -> Jaeger (the second container we started).

3.1 Structured application logs

What is it: Unstructured log: Result saved for user 3. Structured log: pieces stored by NAME: UserId=3, ExamId=1, Score=85. Same information, but structured pieces are searchable. JSON format is simply one way to print them so every field is visible.

51. Replace the plain LogInformation line inside SubmitResult (module 1.4) with the version shown there using named {UserId} {ExamId} {Score} holes - it is already structured.
52. Turn the whole console into JSON: Program.cs, right after CreateBuilder:

```
builder.Logging.AddJsonConsole();
```

53. Restart; submit one result in Postman (fresh key).

You should see:

```
Terminal prints a JSON line with separate fields UserId, ExamId, Score. Rule: good logs
never contain passwords or secrets; fields are named, never glued.
```

3.2 Metrics (easy version) and distributed tracing

What is it: Metrics = numbers on a counter board: requests so far, errors so far, slow calls so far. We skip the heavyweight Prometheus/Grafana setup and make our own counters - same thinking, zero extra containers. Tracing = Jaeger draws each request as a timeline tree; even the call to the fake partner appears as a child step.

Step 1 - counters: create Services/Day3/Counters.cs

```
namespace Practicals.Services.Day3
{
    public static class Counters
    {
        public static long Requests;        // every call
        public static long ServerErrors;    // 5xx answers
    }
}
```

Step 2 - count every request: add this middleware to Program.cs AFTER var app = builder.Build();

```
app.Use(async (ctx, next) =>
{
    Practicals.Services.Day3.Counters.Requests++;
    await next();
```

```
    if (ctx.Response.StatusCode >= 500)
Practicals.Services.Day3.Counters.ServerErrors++;
});
```

Step 3 - the metrics endpoint (Day3Controller):

```
// GET /api/day3/metrics  (our mini "monitoring dashboard")
[HttpGet("metrics")]
public IActionResult Metrics()
{
    var r = Practicals.Services.Day3.Counters.Requests;
    var e = Practicals.Services.Day3.Counters.ServerErrors;
    return Ok(new { requests = r, serverErrors = e,
                    errorRate = r == 0 ? 0 : Math.Round((double)e / r, 4) });
}
```

Step 4 - turn on tracing (Program.cs, with the builder.Services block):

```
builder.Services.AddOpenTelemetry()
    .ConfigureResource(res => res.AddService("Practicals.API"))
    .WithTracing(t => t
        .AddAspNetCoreInstrumentation()   // one trace per incoming request
        .AddHttpClientInstrumentation()   // one sub-trace per outgoing partner call
        .AddOtlpExporter(o => o.Endpoint = new Uri("http://localhost:4317")));
```

Port 4317 goes straight into the jaeger container. The OTel packages are ALREADY in the project - no downloads.

54. Generate traffic: k6 run k6/load.js for a minute.
55. Metrics tab: refresh http://localhost:5000/api/day3/metrics.
56. Traces: browser http://localhost:16686 -> Service Practicals.API -> Find Traces.

You should see:

```
metrics: requests column grows while k6 runs. Jaeger lists every call with its duration
- open any GET /api/Day3/users and see one timeline box.
```

3.3 Correlation IDs and context propagation

What is it: Correlation ID = a case-number for one request: printed in every log it creates, returned in a header so the CALLER can quote it when reporting a problem. Propagation = the number travelling automatically with child calls (the HttpClient instrumentation does this for free with traces).

Demo - extend the counting middleware (replace it):

```
app.Use(async (ctx, next) =>
{
    Practicals.Services.Day3.Counters.Requests++;
    var cid = ctx.Request.Headers["X-Correlation-ID"].FirstOrDefault()
              ?? Guid.NewGuid().ToString("N");
    ctx.Response.Headers["X-Correlation-ID"] = cid;
    var logger = ctx.RequestServices.GetRequiredService<ILogger<Program>>();
```

```
    using (logger.BeginScope(new Dictionary<string, object> { ["CorrelationId"] = cid
}))
    {
        await next();
    }
    if (ctx.Response.StatusCode >= 500)
Practicals.Services.Day3.Counters.ServerErrors++;
});
```

57. Postman: GET /api/day3/users with header X-Correlation-ID: classroom-007.

You should see:

```
Response echoes X-Correlation-ID: classroom-007, and the console log lines carry
CorrelationId: classroom-007. Sentence to memorize: "Give me the ID, I give you the
story."
```

3.4 SLIs and SLOs (reliability targets)

What is it: SLI = what we measure (our errorRate, our p95 latency). SLO = the promise with a deadline (99% success per month; p95 under 300 ms). Error budget = the allowed 1% - spend it on risky releases; when it runs out, ship nothing until health returns.

58. Fill the class table AFTER a load run:

SLI (measured from)	SLO example	Our number today
errorRate in /metrics	< 1%	____
p95 in k6 load run	< 300 ms	____
dotnet test all-green rate	100%	____

You should see:

```
The table filled with real class numbers - that act of writing measured targets IS the
SLO topic done.
```

3.5 API dashboards

What is it: Dashboard = one screen for everyone during an incident. Our offline dashboard = two browser tabs: /metrics (numbers) + Jaeger (timelines). The QUESTIONS it answers are what matter: traffic? errors? slowness? where?

59. Open both tabs side by side. Run load.js. Call these questions aloud as the class reads the tabs: how many requests came? any failures? slowest endpoint?

3.6 Actionable alerts

What is it: Alert = a rule that when broken demands a HUMAN action now. Good alerts fire on user-visible symptoms (errors rising); bad alerts fire on internal trivia (CPU spikes no user feels).

Our paper rule and live trigger:

```
IF  GET /api/day3/metrics shows serverErrors > 0 on two refreshes within 1 minute
THEN 1) open Jaeger, filter errors  2) read the failing trace  3) follow the runbook
(5.13)
```

60. Make it fire: add the deliberate crash endpoint (Day3Controller) and call it 3 times:

```
// GET /api/day3/boom  (on purpose bug for the drill)
[HttpGet("boom")]
public IActionResult Boom() => throw new Exception("exam engine exploded (demo)");
```

You should see:

```
/metrics now shows serverErrors: 3, errorRate > 0 -> our rule "fired": the class walks
to the runbook. That ceremony is alerting - in production a tool automates it.
```

3.7 Troubleshooting 4xx and 5xx errors

What is it: 4xx = the CALLER sent something wrong (their bug). 5xx = OUR code failed (our bug). Different owners, different urgency. Triage order: which endpoint? which call? (correlation id) what does its trace say?

61. From the /boom calls: open the errored trace in Jaeger (Find Traces, look for ! error marks).

You should see:

```
The /boom trace is marked red and stores the exception message. A 404 trace (user typo)
stays calm - caller mistake visible as such.
```

3.8 Diagnosing latency and dependency failures

What is it: Two mysteries observability solves instantly: slow API - was it OUR code or a partner? partner down - where exactly did the call die? Answer: read the trace TREE.

62. Our latency: GET /api/day3/slow?ms=800 -> in Jaeger: ONE 800 ms box, no child - OUR code.
63. Partner slow: first add a call-through endpoint (Day3Controller):

```
// GET /api/day3/publish?mode=ok|slow|fail   (calls the fake partner)
[HttpGet("publish")]
public async Task<IActionResult> Publish([FromServices] IHttpClientFactory http,
                                         [FromQuery] string mode = "ok")
{
    try
    {
        var res = await http.CreateClient()
            .GetAsync($"http://localhost:5000/mock/reportcard?mode={mode}");
        res.EnsureSuccessStatusCode();
        return Content(await res.Content.ReadAsStringAsync(), "application/json");
    }
    catch
```

```
    {
        return StatusCode(502, new { error = "report service failed" });
    }
}
```

64. GET /api/day3/publish?mode=slow -> find its trace. Repeat mode=fail.

You should see:

```
mode=slow: the child box (GET /mock/reportcard) owns the 4 s - partner fault, instantly
visible. mode=fail: child box red + our answer 502. Refresh /metrics: errorRate rose
only when OUR handler answered 5xx.
```

STUDENT EXERCISE - Module 3 wrap

65. Trigger the alert again. This time YOU diagnose: give the trainer only the X-Correlation-ID style answer - trace -> root cause -> suggested fix.
66. Write the Day 3 SLO number into the table (your measured p95 from load.js).

Module 4 - CI/CD & API Deployment (15:15 - 16:45)

4.1 Containerizing API services

What is it: A container image packs: our app + .NET runtime + settings into ONE box that runs identically on any machine. Dockerfile = the recipe. Build it once; run it anywhere.

Why we care: Fixes the oldest excuse in software: "but it worked on my machine".

Offline check FIRST (this decides today's path):

```
docker images | findstr "dotnet"
```

You should see:

```
Two lines naming mcr.microsoft.com/dotnet/sdk:10.0 and .../aspnet:10.0 = build can
happen offline. If MISSING -> skip the docker build below and jump to the two-folders
alternative inside 4.4 (same ideas, no images needed).
```

Create file Practicals/Dockerfile (complete):

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY Practicals.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app --no-restore

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app .
ENV ASPNETCORE_URLS=http://+:8080
ENTRYPOINT ["dotnet", "Practicals.dll"]
```

67. Build and run it (the container reaches SQL Server via host.docker.internal):

```
docker build -t advancedapi:day3 -f Practicals/Dockerfile Practicals
docker run -d --name api-blue -p 5001:8080 `
  -e "ConnectionStrings__Day3Db=Server=host.docker.internal,1433;Database=Day3Db;User
Id=sa;Password=Sa@123456;TrustServerCertificate=True" `
  advancedapi:day3
```

68. curl http://localhost:5001/api/day3/health
69. curl http://localhost:5001/api/day3/users

You should see:

```
OK, then 4 real users - the container IS our API running from an image, reading the
real Day3Db. Everything inside one recipe.
```

4.2 Environment-independent configuration

What is it: Same code, different settings per environment - config comes from OUTSIDE: environment variables override appsettings.json, so nobody edits code to move from dev to staging. ConnectionStrings__Day3Db with double underscores maps to the nested setting.

Demo - two proofs in two minutes:

70. Proof 1 already happened: the container (4.1) got its entire connection string through -e ...ConnectionStrings__Day3Db=... with ZERO config files inside the image.
71. Proof 2 - add an endpoint:

```
// GET /api/day3/config  (reads the environment)
[HttpGet("config")]
public IActionResult ShowConfig(IConfiguration cfg, IWebHostEnvironment env)
    => Ok(new { environment = env.EnvironmentName, who = cfg["Demo:Owner"] ?? "not set"
});
```

72. Run normally: GET /api/day3/config. Then:

```
$env:Demo__Owner = "Team A"
dotnet run --urls http://localhost:5000
```

You should see:

```
First call: who = "not set". Second call: who = "Team A" - same code, config changed
from outside. THIS is environment-independent configuration.
```

4.3 CI pipeline - one script, five gates (build, analysis, tests, security, artifact)

What is it: CI = "Continuous Integration": every change goes through the SAME automatic checker before anyone is allowed to merge/deploy it. No gate, no merge - feelings do not count. Each gate asks one question:

Gate	Question
1 RESTORE	do all packages resolve?
2 BUILD + STATIC ANALYSIS	does it compile cleanly with code-quality analyzers as errors?
3 TESTS	does every test pass?
4 SECURITY	are known-bad packages absent?
5 ARTIFACT	produce the deployable folder/image

Create Practicals/ci-local.ps1 (complete):

```
$ErrorActionPreference = "Stop"

Write-Host "== GATE 1: RESTORE (from the offline cache) =="
dotnet restore Practicals.csproj

Write-Host "== GATE 2: BUILD + STATIC ANALYSIS =="
```

```
dotnet build Practicals.csproj -c Release --no-restore -warnaserror

Write-Host "== GATE 3: TESTS =="
dotnet test Practicals.csproj -c Release --no-build
if ($LASTEXITCODE -ne 0) { throw "GATE 3 FAILED: a test is red" }

Write-Host "== GATE 4: SECURITY REVIEW =="
dotnet list Practicals.csproj package
# NOTE: the online check (--vulnerable) needs internet; offline = read the list aloud.
# In the real (online) pipeline this line becomes: --vulnerable and any hit fails the
gate.

Write-Host "== GATE 5: ARTIFACT =="
dotnet publish Practicals.csproj -c Release -o publish --no-build
Write-Host "== CI GREEN: artifact ready in publish/ =="
```

73. Run it: ./ci-local.ps1
74. Then break one ModelTests assertion on purpose and rerun.

You should see:

```
Green run: all five banners print. Broken run: stops at GATE 3, NO artifact appears.
Restore the test. That stop-the-line behavior is the entire meaning of a quality gate.
```

4.4 CD strategies - rolling, blue-green, canary

Strategy	Plain picture	Strength	Weakness
Rolling	replace instances one by one	cheap, gradual	mixed versions mid-rollout
Blue-green	two complete copies; flip ALL traffic	instant switch + instant rollback	two copies during release
Canary	tiny % of users try the new one first	real evidence before full risk	needs traffic splitting + watching

Simplification: no Kong gateway today. Our "load balancer" = WHICH port we call (5001 = blue/old, 5002 = green/new). The discipline is identical to real tools.

Blue-green demo:

75. Blue running (the api-blue container on 5001, or one published folder on port 5001).
76. Make v2: edit Health endpoint text to "OK v2"; publish to a second folder: dotnet publish -c Release -o publish-v2. Run green on another port:

```
cd publish-v2
$env:ASPNETCORE_URLS = "http://localhost:5002"
dotnet Practicals.dll
```

77. Check both: curl localhost:5001/api/day3/health and localhost:5002/api/day3/health.

78. Cutover: the class switches its Postman URL from 5001 to 5002 (a real LB does this flip for you).
79. Rollback: switch the URL back to 5001 - done in seconds because blue never stopped.

You should see:

```
Health answers v1 on 5001, "OK v2" on 5002 - two live versions, and nobody saw
downtime. The class just performed a blue-green release AND a rollback.
```

Canary demo - 10% of traffic to green (k6 does the split, like a real LB):

```
File Practicals/k6/canary.js (complete):
import http from 'k6/http';
export const options = { vus: 10, duration: '30s' };
export default function () {
  const url = Math.random() < 0.9
    ? 'http://localhost:5001/api/day3/health'    // 90% blue
    : 'http://localhost:5002/api/day3/health';   // 10% green
  http.get(url);
}
```

80. Run k6/canary.js; watch BOTH terminals.

You should see:

```
About 9 of 10 requests appear in blue's console, 1 in 10 in green. If v2 breaks -> send
0% (abort). If clean 30s -> promote: next run points fully at 5002. Watching metrics
while traffic is split IS canary thinking.
```

4.5 Database migration considerations

What is it: While blue and green both live, BOTH talk to ONE database. Rule: any schema change must work for BOTH versions. So: first release ADDS (nullable) -> fill data -> a later release removes the old. Never auto-migrate at container start; make migrations a deliberate, reviewable step.

Why we care: A schema change that old code cannot read = downtime for everyone at once.

81. Generate a reviewable SQL script instead of running blindly (PMC):

```
Script-Migration -Context Practicals.Services.Day3.Day3DbContext -Idempotent
```

You should see:

```
A .sql file with IF-guards appears (idempotent = safe to run twice). Saving this WITH
the release is the professional habit. NOTE: today we do NOT change your existing
Users/ExamDetails/ExamResults tables - all Day 3 work uses them as-is.
```

4.6 Release readiness and rollback

What is it: Readiness = a checklist, written BEFORE the deploy. Rollback = the planned way back, rehearsed while calm. Both are paper habits, not tools.

82. Checklist for the capstone deploy (say each aloud, tick it):

- All five CI gates green (ci-local.ps1 just ran).
- Any schema change reviewed as SQL (4.5).
- Dashboard tabs open (metrics + Jaeger) BEFORE traffic.
- Rollback rehearsed (URL flip done once already).
- Risky new behavior hidden behind a feature flag (4.7).

4.7 Feature flags and change management

What is it: Feature flag = a settings switch that turns new behavior on/off WITHOUT redeploying. Deploy code dark; light it up when the business is ready; kill it in one step if it misbehaves.

Demo:

83. appsettings.json add: "Features": { "ShowExamStats": false }

```
// GET /api/day3/stats - old answer unless the flag is ON
[HttpGet("stats")]
public async Task<IActionResult> Stats(IConfiguration cfg)
{
    int results = await _db.ExamResults.CountAsync();
    if (cfg.GetValue<bool>("Features:ShowExamStats"))
        return Ok(new { results, shape = "v2 dashboard data" });
    return Ok(new { results });
}
```

84. GET /api/day3/stats. Then flip without editing code:

```
$env:Features__ShowExamStats = "true"
# restart the API, call again
```

You should see:

```
First: { results: n }. After the switch: v2 shape. That toggle IS a feature flag -
deployed, but dark, then on, and it can be off again instantly. this is how risky
features ship calmly.
```

STUDENT EXERCISE - Module 4 wrap

85. Do one full rehearsal yourself: build v3 with a new health message, run it at 5002, switch traffic, roll back. Time yourself.
86. Add one bullet to the checklist that YOUR project specifically needs (say it aloud).

Module 5 - API Management & Operations (16:45 - 17:30)

Theory shortcut: normally a GATEWAY (Kong / Azure APIM) is the single front door doing routing, keys, limits, transforms for every API. We keep it simple: ONE middleware does the same jobs inside our API. The ideas transfer 1:1 - names change, concepts do not.

5.1 API Gateway routing and policies

What is it: Gateway = one front door for ALL apis: incoming calls get matched to routes and pushed to the right service, and policies (limits, keys) sit at this door.

87. Whiteboard picture: Caller -> GATEWAY -> [Day1 API, Day2 API, Day3 API]. Policies attached at the arrow.
88. Our local substitute proving mediation: the middleware you already added stamps every response (next topic).

5.2 Request/response transformations

What is it: Transformation = the middleman changes the envelope: add/remove headers, tag versions - services stay untouched.

Demo (add AFTER Build in Program.cs):

```
app.Use(async (ctx, next) =>
{
    ctx.Response.Headers["X-Api-Version"] = "v1";
    await next();
});
```

You should see:

```
Every response now has header X-Api-Version: v1 - "gateway transformation" learned at
middleware level.
```

5.3 Rate limiting, quotas and throttling

What is it: Rate limit = N calls per minute window (burst control). Quota = total per month (business plan). Throttle = make clients wait instead of rejecting. Ours: built-in ASP.NET limiter, no containers.

89. Program.cs before Build:

```
builder.Services.AddRateLimiter(o =>
{
    o.RejectionStatusCode = 429;
    o.AddFixedWindowLimiter("hard5", o2 =>
    { o2.PermitLimit = 5; o2.Window = TimeSpan.FromMinutes(1); });
});
```

90. After Build: app.UseRateLimiter(); Day3Controller endpoint:

```
// GET /api/day3/users-limited (max 5 calls per minute - gateway-style rule)
```

```
[HttpGet("users-limited")]
[Microsoft.AspNetCore.RateLimiting.EnableRateLimiting("hard5")]
public async Task<IActionResult> LimitedUsers()
    => Ok(await _db.Users.CountAsync());
```

91. Call /api/day3/users-limited seven times quickly.

You should see:

```
Calls 1-5 -> 200; call 6 -> 429 Too Many Requests. Wait one minute -> 200 again. Rate
limiting = producing that 429 on purpose protects real APIs from floods.
```

5.4 WAF and schema validation

What is it: WAF (Web Application Firewall) blocks dangerous REQUEST SHAPES before they reach the app - giant bodies, attack strings. Schema validation = inputs must match the OpenAPI contract shape.

92. Edge guard demo (middleware before MapControllers):

```
app.Use(async (ctx, next) =>
{
    if (ctx.Request.ContentLength > 100_000)
    { ctx.Response.StatusCode = 413; await ctx.Response.WriteAsync("request too
large"); return; }
    await next();
});
```

You should see:

```
POSTing a body bigger than ~100 KB -> 413, endpoint code never ran. Plus: our
DataAnnotations already validate the schema (topic 1.6 negative matrix) - two layers of
the same shield.
```

5.5 API analytics and usage monitoring

What is it: Analytics = numbers that answer "who calls what, how much, with how many failures". Ours is one endpoint: /api/day3/metrics (requests, serverErrors, errorRate).

93. After a k6 run: GET /api/day3/metrics and write the numbers into the SLO table (3.4).

You should see:

```
Requests climbed exactly by traffic; errors only after deliberately broken calls. That
split = usage + health monitoring working.
```

5.6 Developer portals

What is it: Portal = the shop window for API consumers: catalogue, docs, try-it, keys. Ours simply IS Swagger + the baseline JSON + this guide - everything a consumer needs to start alone.

Portal feature	What we use today
Catalog of endpoints	/swagger
Machine-readable contract	/openapi/v1.json
Try-it console	Swagger Try it out
Onboarding story	Module 0 of this guide

5.7 API documentation and onboarding

What is it: Documentation works when a stranger gets their FIRST successful call in minutes - alone. This is measured with a clock, not vibes.

94. Pair drill: Partner sees ONLY base URL http://localhost:5000 and Swagger. Goal: call GET /api/day3/exams successfully. Note the time and every stuck moment.

You should see:

```
Under 5 minutes. Every stuck moment = a fix to add to docs (透明度 not elastic charm).
```

5.8 SDK considerations

What is it: SDK = a ready-made client library so consumers write api.GetExams() instead of raw HTTP calls. Costs maintenance per language - generate from the OpenAPI spec, publish only when consumers really exist.

95. Postman: open GET /api/day3/exams -> the </> Code panel on the right -> choose C# (RestSharp).

You should see:

```
A complete C# client snippet generated instantly, offline. Discussion: when does an SDK
pay off? (many consumers, hard auth).
```

5.9 Sandbox environments

What is it: Sandbox = the practice area: real-ish API, fake or scratch data, zero consequences. Our /mock/reportcard is exactly such a sandbox for the partner; a second instance with a scratch database is a sandbox for consumers.

96. Talk it through: how would you give an extern a sandbox of ours? (Second instance: another port + ConnectionStrings__Day3Db pointing to Day3Sandbox. No real data harmed.)

5.10 API ownership and governance

What is it: Every API needs a named OWNER (a team, not "everyone"), rules everybody agreed on (style guide), and a LIST of all APIs (catalogue) with their status.

97. Create Practicals/API-CATALOG.md:

```
| API                | Owner  | Version | Status  | Contract              |
|--------------------|--------|---------|---------|-----------------------|
| Day3 Exams/Results | Team A | v1      | Active  | /openapi/v1.json      |
| Mock Report Service| Team B | v1      | Sandbox | /mock/reportcard      |
```

Governance questions for the class: who approves a new endpoint? who owns the alert from 3.6? Write the chosen answers INTO this file.

5.11 Backward compatibility and API retirement

What is it: Compatible = add fields/endpoints (old callers untouched). Breaking = rename/remove/change types -> that forces a NEW version alongside the old. Retirement = announce -> mark deprecated -> watch use -> sunset date -> remove.

Demo (two versions + retirement headers):

```
// GET /api/day3/v1/exams  (old shape, now deprecated)
[HttpGet("v1/exams")]
public async Task<IActionResult> ExamsV1()
{
    Response.Headers["Deprecation"] = "true";
    Response.Headers["Sunset"] = "Sat, 01 Aug 2026 00:00:00 GMT";
    return Ok(await _db.ExamDetails.AsNoTracking()
        .Select(e => new { e.Id, e.ExamName }).ToListAsync());
}

// GET /api/day3/v2/exams  (new shape)
[HttpGet("v2/exams")]
public async Task<IActionResult> ExamsV2()
    => Ok(new { version = "2.0",
                data = await _db.ExamDetails.AsNoTracking().ToListAsync() });
```

You should see:

```
v1 answers with headers Deprecation: true and a Sunset date - tools and SDKs can auto-
warn. v2 answers the new envelope. Both live peacefully until sunset day.
```

5.12 Incident response and operational runbooks

What is it: Incidents at 2 AM: nobody thinks clearly. A runbook = symptom -> numbered steps written calmly in daylight. Process: detect -> triage -> MITIGATE FIRST -> fix -> post-mortem.

Create Practicals/Day3-Runbook.md:

```
# Day3 API Runbook
Owner: Team A | Alert rule: /metrics serverErrors > 0 checked twice within a minute
```

```
## Symptom: 5xx spike
1. Open /api/day3/metrics - errorRate climbing?
2. Open Jaeger http://localhost:16686, service Practicals.API, click error traces.
3. Use the X-Correlation-ID of a failing call to find its log lines.
4. DB suspect? Check SSMS + docker compose ps sqlserver.
5. MITIGATE: flip traffic back to blue (topic 4.4) - then fix forward.

## Symptom: high latency
1. /metrics + Jaeger durations - which endpoint?
2. Child step slow = partner; our step slow with SQL = database.
3. Fixes: cache (2.7), page (2.8), move work to a job (2.9).
```

98. Live drill: 5 x GET /api/day3/boom, watch metrics fire the rule, walk the runbook out loud; mitigation = students point their Postman at the still-healthy blue instance.

You should see:

```
Incident mitigated by a rehearsed move, not panic. Relief is the lesson.
```

Capstone - run the whole loop alone (last 30 min)

99. CODE: add GET /api/day3/leaderboard - exam results ordered by score desc with user/exam names (try: query ExamResults join Users and ExamDetails, project to a small anonymous record).
100. TEST: add one Fact for it returning 200 and one asserting the body contains "85".
101. LOAD: k6 against it with the threshold passing.
102. OBSERVE: find its trace in Jaeger; refresh /metrics (numbers grew).
103. SHIP: ci-local.ps1 all gates green; publish as v2, run at 5002, canary-split for 30 s, then promote.
104. OPERATE: fire /boom twice (alert rule triggers), then follow the runbook and roll traffic back to blue.
105. SAY IT: today we measured quality before release, measured speed, watched it run, shipped safely, and knew exactly how to undo - that IS Day 3.

Appendix - Program.cs additions today (checklist)

```
builder.Services.AddDbContext<Day3...>(...)          // Module 0
builder.Services.AddMemoryCache();                    // Module 0
builder.Services.AddHttpClient();                     // Module 0
builder.Services.AddResponseCompression();            // 2.8
builder.Services.AddSingleton<JobService>();          // 2.9
builder.Logging.AddJsonConsole();                     // 3.1
builder.Services.AddOpenTelemetry() ... :4317         // 3.2
builder.Services.AddRateLimiter(...hard5...);         // 5.3
app.Use(counter + correlation middleware);            // 1.3? -> 3.2/3.3
app.UseResponseCompression();                         // 2.8
app.Use(version header stamp);                        // 5.2
```

```
app.Use(body-size guard);                    // 5.4
app.UseRateLimiter();                        // 5.3
public partial class Program { }             // 1.3
```

Troubleshooting

Symptom	Cause	Fix
dotnet test cannot find Program	partial line missing	public partial class Program { } at the very END of Program.cs
User_2_is_Ishan fails	Day3Db not started or connection string typo	docker compose up -d sqlserver; recheck appsettings.json string
No traces in Jaeger	wrong port or jaeger off	endpoint must be http://localhost:4317; docker compose up -d jaeger
429 during my own testing	your own 5/minute limiter	call non-limited endpoint or wait a minute
k6 not recognized	k6 not installed/PATH	run from its folder; re-check installation per startup guide
docker build pulls and fails	dotnet base images absent	use the two-publish-folders alternative in 4.4 - same learning, no images
duplicate-key 500 on results	same student+exam pair twice	correct behavior! The X-Replay path caught it (1.4)

Deliberately skipped (simplicity rules)

- Kong (rate limits & transforms done in middleware), WireMock (fake partner endpoint), Prometheus/Grafana/OTel collector (own /metrics + direct Jaeger).
- Pact-style tooling (consumer-contract file + snapshot instead), real load balancer canary (k6 splitter, honestly labeled).
- The entire course ran with TWO containers and zero internet.